Chapter 3

CPU Scheduling

Outline

- Introduction to scheduling
- Process behavior
- When to schedule
- Categories of scheduling algorithms
- Scheduling algorithms goals (criteria)
- Scheduling algorithms
  - Scheduling in batch systems
  - Scheduling in interactive systems
  - Scheduling in Real time systems

CPU scheduling

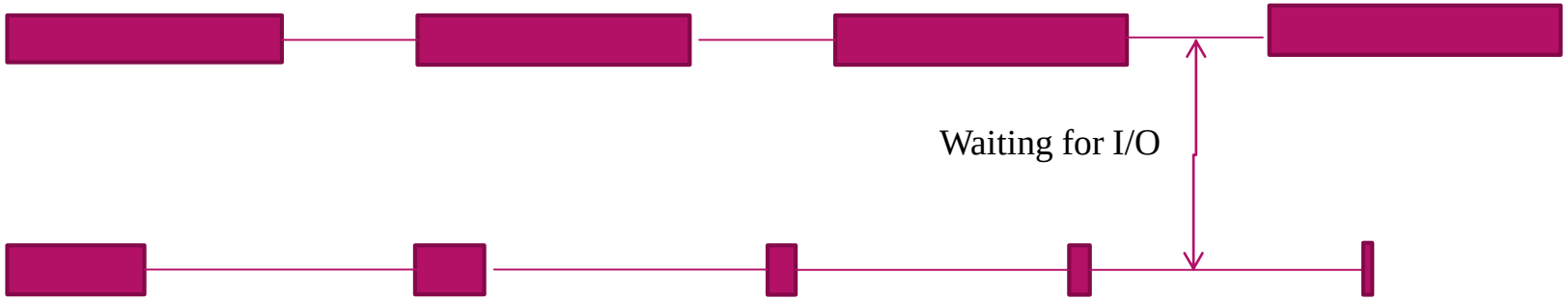
- The aim of process scheduling is to assign processes to be executed by the processor or processors over time in a way that meets the system objectives.
- It ensures optimal CPU utilization, responsiveness, and fairness among competing processes.
- Fundamentally, scheduling is a matter of managing queues to minimize delays and to optimize performance.
- Scheduling is the basis of multi-programmed operating system.
- Some main frame system- use both ways
  - The scheduler decides whether a batch job or interactive user at the terminal should go next.

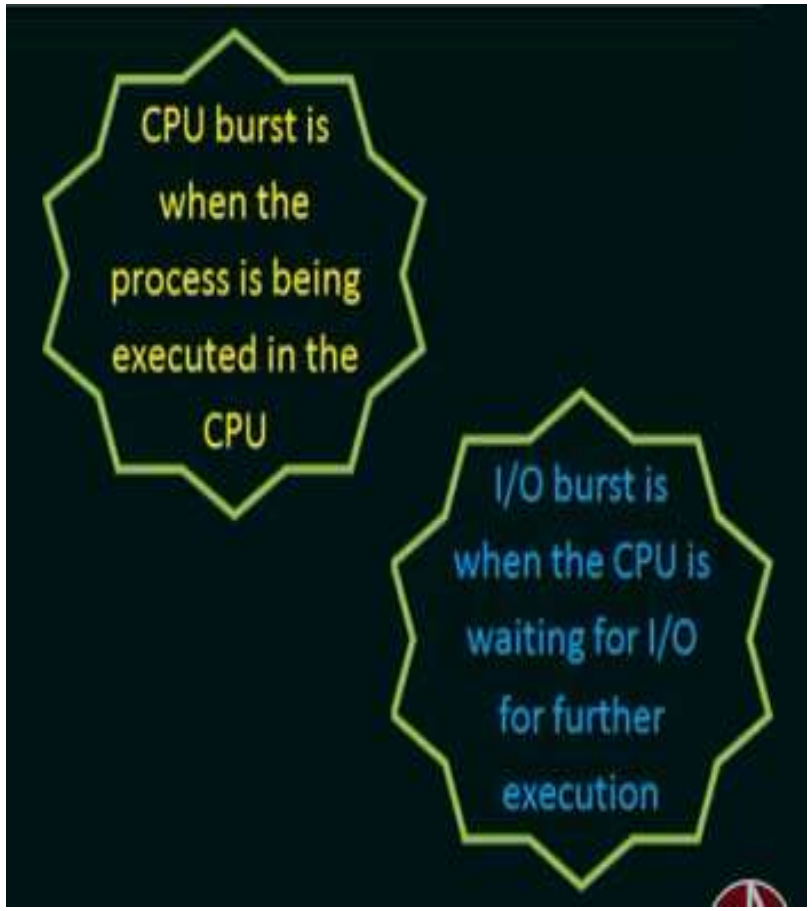
CPU scheduling

- CPU was a scarce resources on these machines. Hence, a good algorithm can make a huge difference.
- But with the advent of personal computer
  - One process at a time
  - CPU is a rarely scarce resource – PC are limited by user input and application type
  - The situation is different when it comes to high-end networked workstations and servers. – sending e-mail and closing windows
- The scheduler has to worry about making efficient use of CPU because switching is expensive.

Process Behavior

- CPU bound – long CPU burst
- I/O bound – long I/O burst





- The key factor is CPU burst.
  - As the CPU gets faster processes tend to get more I/O.
  - And CPU is improving in much faster way than disks.

When to schedule

- Key issues to make scheduling decisions
  - When a new processes are created.
  - When a process switches from running state to waiting state.
    - I/O or an invocation of wait() for the termination of one of the child process.
  - When a process switches from running state to the ready state.
    - When an interrupt occurs.
  - When a process switches from waiting to ready state.
    - Completion of I/O.
  - When a process terminates.

When to schedule

- If a hardware provides periodic interrupts, scheduling decision can be made at each clock interrupts.
  - Non-Preemptive and preemptive
    - Under **non-preemptive** scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state. Otherwise, it is **preemptive**.

Categories of scheduling algorithms

- In different environment different algorithms are implemented
  - Batch systems
    - No user waiting impatiently. Hence, Non-preemptive or long time run preemptive are acceptable
  - Interactive systems
    - Need preemption
    - Most are general purpose – even malicious software can run.
  - Real-Time systems
    - Preemption is oddly enough, some times not needed
    - Run only programs that are needed for further application at hand

CPU scheduling performance metrics

- **Processor Utilization →** The ratio of busy time of the processor to the total time passes for processes to finish.

Processor Utilization = (Processor busy time) / (Processor busy time + Processor idle time)

- **Throughput →** Number of processes that complete their execution per time unit.

Throughput = (Number of processes completed) / (Time Unit)

- **Turnaround time →** Amount of time to execute a particular process.

tat = t(waiting time) + t(burst time)

CPU scheduling performance metrics

- **Waiting time →** Amount of time a process has been waiting in the ready queue.

wt = t(tat time) - t(burst time)

- **Burst Time →** Time of execution, when process is in Processor.
- **Response time →** Amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment).

rt = t(first response) – t(submission of request)

Scheduling algorithms Goals

- All systems
  - **Fairness**
    - Fairness is important. This prevents any single process from monopolizing the CPU
    - Comparable processes should get fair share
  - Enforcing systems policy
    - Safety control and payroll applications in a nuclear reactor
  - **Resource Utilization**
    - Keeping all parts of the computer busy
    - I/O bound and CPU- bound can be run together.

Scheduling algorithms Goals…

- Batch systems
  - Throughput - maximum jobs per unit of time.
  - Turn around time – minimize time between submission and termination
  - CPU utilization – keep the CPU busy all the time
- Interactive systems
  - Response time – respond to request quickly
  - Proportionality –meet users expectations.

Scheduling algorithms Goals…

- Real-Time
  - Meeting deadlines – avoid losing of data
  - Predictability – avoid quality degradation in multi-media systems

Scheduling in Batch systems

- FCFS(First Come First Served)
- SJF(Shortest Job First)
- Non-Pre-emptive Priority

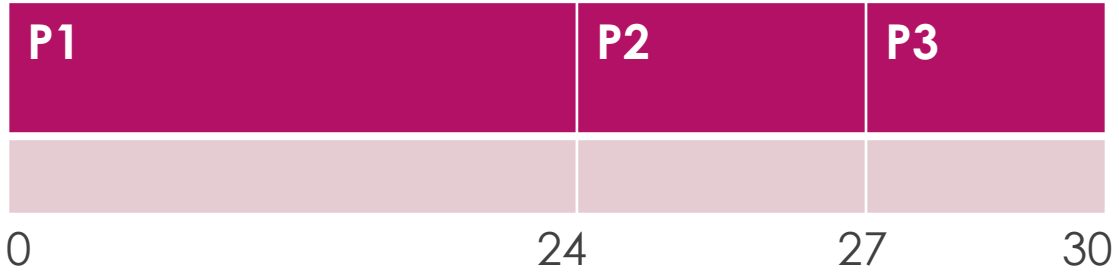
First Come First Served

- The process that requests the CPU first is allocated the CPU first.
- It's the simplest of all algorithms
- It's non-preemptive
- A blocked process come to an end of the queue when it's ready to be executed
- Easy to understand and to program
- A single linked list keeps track of all ready state processes
- The greater disadvantage, it may lead to long waiting times for shorter jobs if longer jobs arrive first(convoy effect).

First Come First Served

Process	Burst Time *(milliseconds)*	*Arrival time*
P1	24	0
P2	3	0
P3	3	0

- Suppose that the processes arrive in the order: *P1* , *P2* , *P3*
- The Gantt Chart for the schedule is:



- Waiting time for *P1* = 0; *P2* = 24; *P3* = 27
- Average waiting time: (0 + 24 + 27) / 3 = 17 *(milliseconds)*
- Average turn-around time: (24 + 27 + 30)/3 = 27

Shortest Job First

- Another non-preemptive and the run time of the processes are known in advance.
- When multiple batch jobs are sitting in a queue with the same priority, the scheduler runs the shortest job first.
- SJF is optimal only if all jobs are available simultaneously.

Short Job First…

Process	Arrival Time	Burst Time
P1	0.0	6
P2	0.0	4
P3	0.0	1
P4	0.0	5

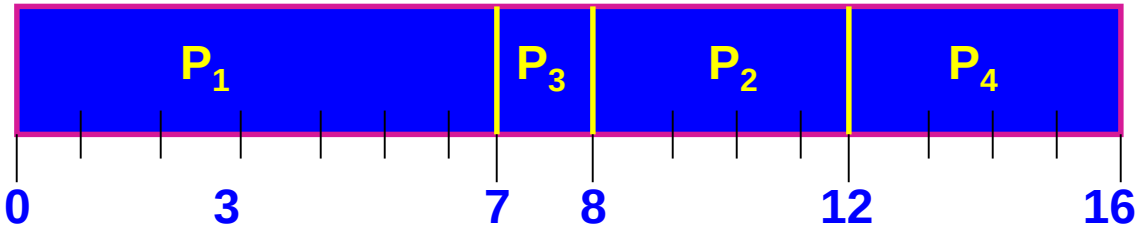
- SJF (non-preemptive, simultaneous arrival)

- Average waiting time = (0 + 1 + 5 + 10)/4 = 4
- Average turn-around time = (1 + 5 + 10 + 16)/4 = 8

Non-Pre-emptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)



Non-Pre-emptive Priority scheduling

- Each process is assigned a priority.
- The process with the highest priority is allowed to run.
- This idea is critical in a multi-user environment and also in a single user with several applications running at a time.
- Priority scheduling may be preemptive or non-preemptive.
- In **non-preemptive priority scheduling**, the currently executing process continues until it voluntarily relinquishes the CPU or completes its execution.

Priority scheduling…

- Unix system has *nice* which allows the user to voluntarily reduce its priority level.
- Process priority can be given statically and dynamically
  - Statically-Role and privilege assignment in MIS systems
  - Dynamically- I/O bound processes should be given the highest priority after waiting for I/O response and asks to run again.
    - This avoid unnecessary memory hold time
    - A simple algorithm for giving this service is to set the priority to *1/f* *where f is fraction of the last quantum that the process used.*
    - *For example, a process that uses 1 msec of its 50 msec quantum then it will be given priority level equivalent to its quantum.*

Priority scheduling….

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

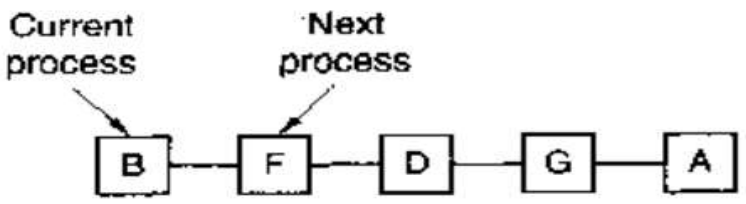
Scheduling in interactive systems

Scheduling in interactive systems

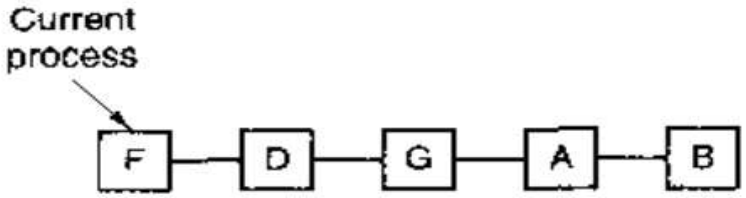
- Round robin scheduling
- Shortest Remaining Time Next
- Priority (Preemptive)
- Multiple queuing
- Shortest process next
- Guaranteed scheduling

Round- Robin scheduling

- Processes will be given equal priority
- It uses quantum- slice of time
- Therefore each process will be given an interval of time (quanta of time).
  - 1 quanta, 2 quanta…..
- Processes are allowed to run in their given slice of time.
- The main issue in round- robin is time taken by context switching. E.g. 1msec. For process switching and 4msec as a quanta



(a)



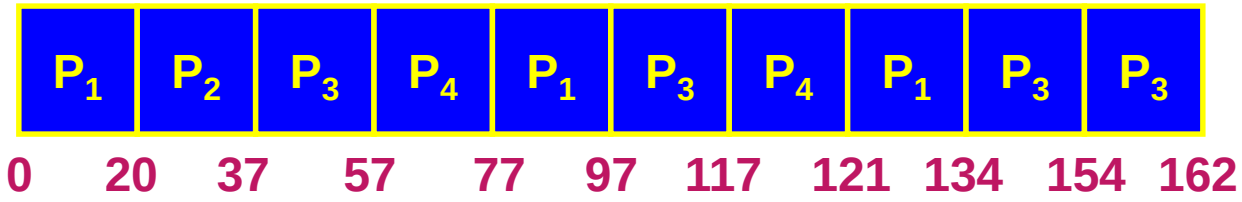
(b)

Round- Robin…

- The two main things as a summary
  - Setting the quantum too short causes to many process switches that will result in performance degradation.
  - But setting it too long may cause poor response to short interactive commands.

Round-Robin….

Process	Burst Time	Q=20
P_1	53	
P_2	17	
P_3	68	
P_4	24	



- Typically, higher average turnaround than SJF, but better *response*.

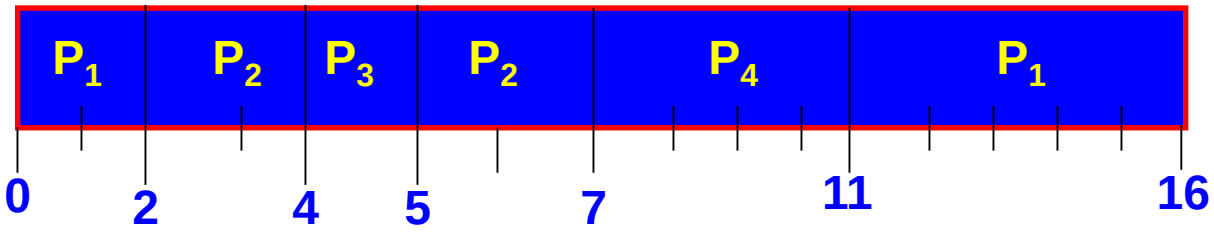
Short remaining time first (Pre-emptive SJF)

- Preemptive version of SJF.
- The scheduler always picks the processes whose remaining running time is the shortest.
- Again the run time should be known in advance.
- The shortest new job has a great advantage over all processes.

Pre-emptive SJF = Short remaining time first

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



Shortest Remaining Time First

Process ID	Arrival Time	Burst Time	Turn Around Time	Waiting Time	Response Time
1	0	8	20	12	0
2	1	4	9	5	0
3	2	2	2	0	0
4	3	1	2	1	1
5	4	3	9	6	6
6	5	2	2	0	0

Shortest Remaining Time First

P1	P2	P3	P3	P4	P6	P2	P5	P1

0 1 2 3 4 5 7 10 13 20

Priority scheduling

- Each process is assigned a priority.
- The process with the highest priority is allowed to run.
- This idea is critical in a multi-user environment and also in a single user with several applications running at a time.
- In **preemptive priority scheduling**, the currently executing process can be preempted if a higher-priority process becomes available for execution.

Priority scheduling….

Process	Arrival Time	Burst Time	Priority
P1	0 ms	3 ms	3
P2	1 ms	4 ms	2
P3	2 ms	6 ms	4
P4	3 ms	4 ms	6
P5	5 ms	2 ms	10

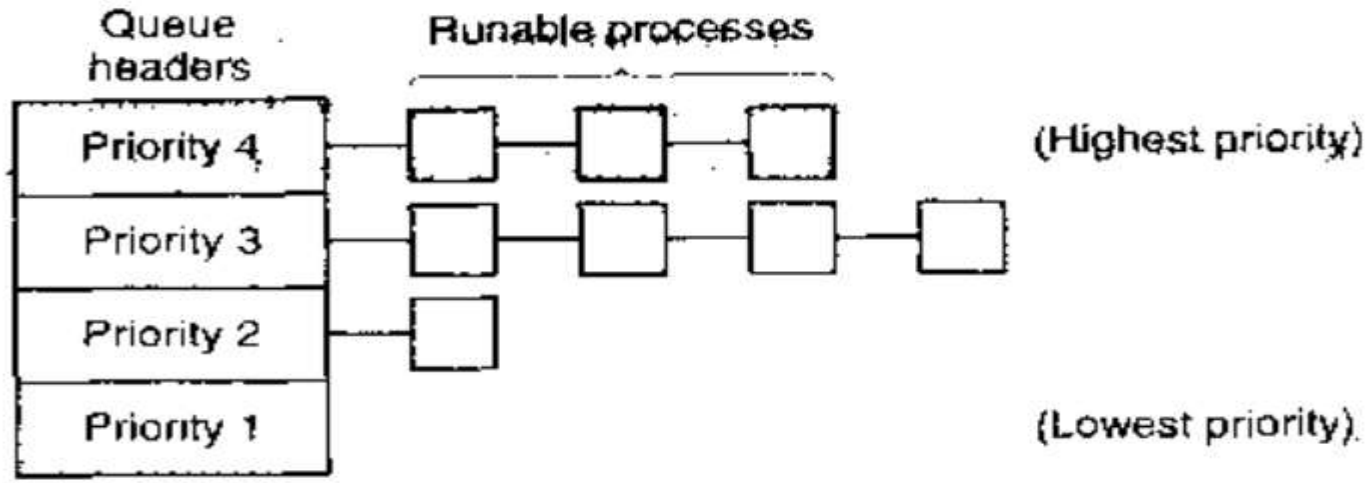
Priority scheduling….

P1	P2	P2	P2	P1	P3	P4	P5

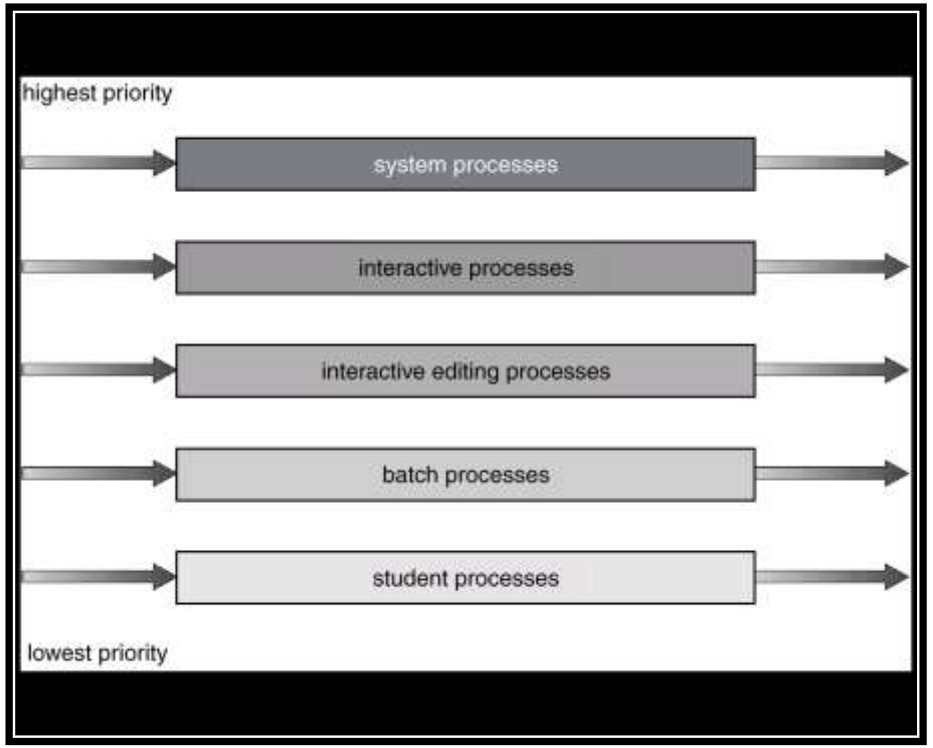
0 1 2 3 5 7 13 17 19

Multiple Queues

- Processes in the highest priority class will run for 1 quantum and processes in the next highest class will run for 2 quanta processes in the next class will run for 4 quanta and so on.
  - Consider how a process that needs100 quanta to continuously run and finish its computation. It would only need 7 swaps unlike 100 swaps in pure round- robin.



Multiple Queues….. Priority level examples



Multiple queues….

- As a processes sank deep and deeper in to the priority queues it would be run less and less frequently.
  - This saves the CPU for short and interactive processes.
- A policy of making a process that was decided to run for a long time to become interactive later.
  - E.g. copying a file or installing a memory demanding software

Guaranteed Scheduling

- Make real promises to the users about performance and live up to them.
  - If n users logged in each will get 1/n of CPU power.
  - If a user have n processes to run each will get 1/n of the CPU cycle.
- To make good on the promise, The system must keep track of how much CPU the process has had since creation.
  - i.e. ratio of actual CPU time consumed to CPU entitled.
  - 0.5, 2.0 ……
  - The algorithm then runs the process with the lowest ratio.

Real-Time Scheduling

- ***Hard real-time systems*** – required to complete a critical task within a guaranteed amount of time.
  - Resource reservation – knows how much time it requires and will be scheduled if it can be guaranteed that amount of time
  - Requires special purpose software running on hardware dedicated to their critical process
- ***Soft real-time computing*** – requires that critical processes receive priority over less fortunate ones.
  - System must have priority scheduling where real time processes are given the highest priority (no aging to keep priorities constant)
  - Dispatch latency must be small – need to be able to preempt system calls particularly in long, complex calls at safe points.

Reading Assignment

- Algorithm evaluation
- Scheduling in different OS

  windows

  Solaris

  Unix

  Mobile OS

*******End of Chapter 3**********